

AMDP 練習 6：AMDP 除錯與資料預覽

Lecture

寫到這裡，你已經踩過好幾次「SQLScript 寫錯、啟用失敗、看編譯器錯誤訊息修正」的循環（am03 的 **FOR...AS**、am04 的 **USING** 逗號、am05 的 **SIGNAL** 條件宣告）。但編譯期抓不到的問題呢？例如「這段 **WHERE** 條件邏輯上對，但撈出來的資料筆數跟預期不一樣」——這種情況需要**執行期**的除錯工具，而不是語法檢查。

一般 **ABAP** 的外部斷點對 **AMDP** 沒有用：AMDP Method 呼叫進去之後，實際執行的是 HANA 資料庫裡的一個 Database Procedure，不是 ABAP 應用伺服器裡的程式碼——ABAP 偵錯器只能在 Method 呼叫前、呼叫後設中斷點，沒辦法「跳進去」SQLScript 本體單步執行，因為那段程式碼根本不是在 ABAP 執行環境裡跑的。

SAP 為此提供了專用的 **AMDP Debugger**（Eclipse ADT 內建功能，這套系統的 ADT Discovery 已確認有 </sap/bc/adt/amdp/debugger/main> 這個資源），可以直接在 SQLScript 原始碼裡設中斷點、單步執行、檢視變數值——用法概念跟一般 ABAP 偵錯器很像，只是它是連到 HANA 的除錯協定，不是 ABAP 應用伺服器的除錯協定。另外還有 **Data Preview**（</sap/bc/adt/datapreview/amdp>），可以不用寫程式就直接看某張表或某個 AMDP 相關物件的實際資料內容，適合開發過程中快速確認「這張表現在長什麼樣子」。

學習目標

- 理解為什麼一般 ABAP 偵錯器沒辦法單步執行 AMDP 的 SQLScript 本體（執行環境不同：ABAP 應用伺服器 vs HANA 資料庫引擎）
- 知道 Eclipse ADT 有專用的 **AMDP Debugger**，可以在 SQLScript 原始碼裡設中斷點、單步偵錯——這是 Eclipse 裡的互動功能，這門課的 Claude 端工具（ADT REST API）沒辦法自動操作，需要你自己在 Eclipse 動手試
- 學會用 **Data Preview** 快速查看一張表的實際資料，不用寫一支程式、不用打 SQL，適合開發時快速確認資料現況（這門課第一次沒有走 ABAP 程式碼、直接用 IDE 工具查資料）

事前準備

ADT 端物件已由課程準備好（**\$TMP**）：

- ZR_AM06_DEMO**——demo 程式，呼叫 am04 已經驗證過的 **ZCL_AM04_ROUTE_LOAD=>GET_ROUTE_LOAD_FACTOR**，用經典 **ALV Function Module REUSE_ALV_GRID_DISPLAY** 顯示結果——這題沒有新的 **AMDP** 類別，刻意重用 am04 已驗證正確的方法，讓這題可以專心練習「怎麼在 Eclipse 里對一個既有的 AMDP 呼叫設中斷點」，不用同時應付新的 SQLScript 語法

題目需求

1. **ZR_AM06_DEMO**：

```
``abap REPORT zr_am06_demo.
```

```
START-OF-SELECTION.
```

```
zcl_am04_route_load=>get_route_load_factor( EXPORTING iv_mandt = sy-mandt IMPORTING et_routes = DATA(lt_routes) ).
```

```
DATA: lt_fieldcat TYPE slis_t_fieldcat_alv, ls_fieldcat TYPE slis_fieldcat_alv.
```

```
DEFINE add_fieldcat. CLEAR ls_fieldcat. ls_fieldcat-fieldname = &1. ls_fieldcat-seltext_m = &2. ls_fieldcat-outputlen = &3. APPEND ls_fieldcat TO lt_fieldcat. END-OF-DEFINITION.
```

```
add_fieldcat: 'CARRID' 'Carrier' 3, 'CARRNAME' 'Carrier Name' 20, 'CONNID' 'Connection' 4, 'FLIGHT_CNT' 'Flights' 6, 'SEATS_OCC' 'Seats Occ.' 8, 'SEATS_MAX' 'Seats Max' 8, 'LOAD_PCT' 'Load %' 6.
```

```
CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY' EXPORTING i_callback_program = sy-repid it_fieldcat = lt_fieldcat TABLES t_outtab = lt_routes. ``
```

這題的輸出改用經典 **ALV** (**REUSE_ALV_GRID_DISPLAY** 這個標準 Function Module)，跟 am02/am06 草稿階段原本用的 **cl_salv_table** (Functional ALV，承 OOP op11) 是兩種不同世代的 ALV API：

- **cl_salv_table** (**Functional/OO ALV**)： **FACTORY**() 靜態工廠方法建立物件，欄位資訊自動從 Internal Table 的執行期型別 (RTTI) 推導，不用手動組欄位目錄，是 SAP 目前推薦的寫法 (op11 教過)
- **REUSE_ALV_GRID_DISPLAY** (**經典 ALV**)：年代更久、大量存在於既有系統的舊程式碼裡，因為這題的 **lt_routes** 是 AMDP 方法裡宣告的本地 **TYPES** (不是 DDIC 註冊過的結構)，沒辦法用 **I_STRUCTURE_NAME** 參數自動產生欄位目錄，**要手動組一份 IT_FIELDCAT** (**SLIS_T_FIELDCAT_ALV** 表格，每個欄位手動指定 **FIELDNAME/SELTEXT_M/OUTPUTLEN**)——這是這題特別想讓你對照的地方：Functional ALV 靠型別反射自動化掉的事情，經典 ALV 要自己動手做

兩者都能正常顯示 AMDP 回傳的 Internal Table，選哪一種純粹是程式碼世代/團隊慣例的差異，不影響跟 AMDP 的整合方式。

1. 在 Eclipse ADT 裡實際操作 AMDP Debugger (本題唯一需要你在 Eclipse 手動操作的步驟)：

1. 在 Eclipse ADT 打開 **ZCL_AM04_ROUTE_LOAD** 的 **GET_ROUTE_LOAD_FACTOR** 方法原始碼 (SQLScript 本體那一段)
2. 在 SQLScript 程式碼的某一行 (例如 **GROUP BY f.mandt, f.carrid, f.connid** 那一行) 點選左邊界設一個中斷點——這個中斷點圖示看起來跟一般 ABAP 中斷點類似，但這是 **AMDP 專用的中斷點**
3. 執行 **ZR_AM06_DEMO** (F8)，執行到這個 AMDP 呼叫時，Eclipse 應該會跳出詢問是否要切換到 AMDP Debugger 透視圖 (perspective) 並停在中斷點——如果沒有跳出，檢查你的使用者是否有 AMDP 除錯需要的授權 (通常需要額外的除錯相關權限，跟一般 ABAP 除錯權限不完全一樣)
4. 停在中斷點後，可以看到目前 SQLScript 執行到哪一行、目前的變數/中間表格內容——這是一般 ABAP 偵錯器完全看不到的東西
5. 單步執行 (Step Over) 到 **et_routes = ...** 那一段，觀察最終組出來的結果

1. 用 Data Preview 快速查看 **SFLIGHT** 現況 (不用寫程式)：

在 Eclipse ADT 裡開啟 **SFLIGHT** 表的 Data Preview (右鍵 → Open With → Data Preview，或直接在 Project Explorer 對表按右鍵選 Data Preview)，可以直接看到目前的資料列，不用寫任何 **SELECT** 程式——這個技巧在這門課開發過程中其實已經用過很多次 (例如 am01 診斷「同一個 carrid 重複出現」時，就是先看資料現況

才發現多 Client 的問題)，只是之前是透過指令列直接打 ADT 的 Data Preview API，你在 Eclipse 裡用滑鼠點幾下就能做到一樣的事

預期輸出

ZR_AM06_DEMO 執行後跳出一個 ALV 畫面，欄位跟 am04 的 ZCL_AM04_ROUTE_LOAD 輸出一致 (CARRID/CARRNAME/CONNID/FLIGHT_CNT/SEATS_OCC/SEATS_MAX/LOAD_PCT)，資料內容也跟 am04 實測畫面相同 (因為呼叫的是同一個已驗證的 AMDP 方法)。

AMDP Debugger 的畫面效果需要你在 Eclipse 實際操作才能看到——這部分無法用文字預先告訴你「應該長怎樣」，因為每次停在中斷點看到的變數值會依當下資料而定，這也是除錯工具本來的用途：讓你自己觀察，而不是照抄一個固定答案。

團隊實務備註

- 這題是這門課第一次「本體程式碼」沒有新東西、純粹練習工具操作——ZR_AM06_DEMO 呼叫的 AMDP 方法完全沿用 am04，這是刻意的：除錯工具的練習不應該同時綁著「還要弄懂新語法」，用一個已經確定正確的例子來練習「怎麼觀察它的執行過程」，學習效果比較純粹
- REUSE_ALV_GRID_DISPLAY 這種會產生全螢幕 ALV Grid 畫面的呼叫，一樣沒辦法透過 ADT 的無頭 programrun API 自動驗證——跟 am02 遇到的 cl_salv_table->display() 是同一種限制 (都需要真實 GUI 前端)，這題只驗證到「語法檢查/啟用成功、lt_routes 資料邏輯沿用 am04 已驗證正確」，實際 ALV 畫面效果一樣要你在 SAP GUI 執行 ZR_AM06_DEMO (F8) 親眼確認
- 手動組 IT_FIELDCAT 時 OUTPUTLEN 沒設或設太小，欄位內容可能被截斷顯示——這題的 CARRNAME (航空公司全名，最長 20 字元) 特別給了 OUTPUTLEN = 20，如果沿用其他欄位的短長度，畫面上長公司名稱可能顯示不全，這是手動組欄位目錄時容易漏掉的細節 (cl_salv_table 因為有型別反射，通常不會有這個問題)
- AMDP Debugger 需要的權限，通常跟一般 ABAP 除錯權限 (S_DEVELOP 之類) 不完全一樣，如果 Eclipse 沒有跳出 AMDP Debugger 的提示，第一個該檢查的不是程式碼有沒有問題，而是使用者權限——這是一個常見的「明明步驟做對了，但工具沒反應」的踩坑點，只是這次沒有在開發過程中實際踩到 (因為 Claude 端沒有 Eclipse GUI 環境可以測試這一段)，如果你在 SAP GUI/Eclipse 操作時遇到跳不出偵錯畫面，優先確認權限而不是懷疑自己操作錯誤
- Data Preview 這個技巧不限於 AMDP 相關的表，前面所有課程 (REST/OOP) 用到的 SFLIGHT/SCARR/SBOOK 都可以用同樣方式快速預覽，開發時比自己寫一支 SELECT 程式再執行快很多

思考題

1. 如果 AMDP Debugger 停在中斷點時，你想知道某個中間 Table Variable (例如 am04 的 route_totals CTE) 目前的內容，偵錯器介面上應該去哪裡找？ (提示：對照一般 ABAP 偵錯器的「變數」窗格，AMDP Debugger 通常也有類似的區域可以檢視表格型變數的內容，不只是純量變數)
2. 這門課從 am01 到 am05，每次踩到 SQLScript 語法問題都是透過「啟用失敗 → 看編譯器錯誤訊息 → 修正」找出來的——這種方式抓得到「語法錯誤」，但抓不到「語法正確、邏輯卻不是你想要的」這種問題 (例如 am01 一開始沒過濾 Client、資料重複，如果不是拿到執行結果實際比對，光看語法檢查是抓不出來的)。這算不算是「需要用到 Debugger / Data Preview」的情境？為什麼前五題都沒有真的動用到偵錯器就抓出問題了？

答案

見 `zr_am06_demo.prog.abap` (SAP 端物件 `ZR_AM06_DEMO` , 套件 `$TMP`) 。 AMDP 方法沿用 `am04` 的 `ZCL_AM04_ROUTE_LOAD` (`zcl_am04_route_load.clas.abap`) 。 AMDP Debugger 操作步驟請在 Eclipse ADT 實際跟著上方步驟操作 , 這部分無法由 Claude 端自動驗證。